

Examen

20/20 points

Question 1 (1.00 points)

D'où provient le mot "Algorithme" et quelle est la définition générale d'un algorithme d'après le cours ? (Sélectionnez toutes les affirmations correctes)

- Le mot dérive du nom du mathématicien Al-Khwarizmi qui a vécu au 9ème siècle. (correcte)
- Un algorithme est intrinsèquement dépendant du langage de programmation dans lequel il est implémenté.
- Un algorithme prend des données en entrée, exprime un traitement particulier et fournit des données en sortie. (correcte)
- Un algorithme est une série d'instructions exécutées en parallèle sur un matériel informatique spécifique.

Question 2 (3.00 points)

Le cours identifie trois problèmes fondamentaux en algorithmique : la Complexité, la Calculabilité et la Correction. Définissez brièvement chacun de ces trois concepts d'après les explications du cours.

Réponse correcte : D'après le cours, voici les définitions des trois problèmes fondamentaux :

1. **Complexité** : Concerne les performances et les ressources requises par un algorithme. Elle répond aux questions : En combien de temps un algorithme va-t-il atteindre le résultat escompté ? Et de quel espace (mémoire) a-t-il besoin ?
2. **Calculabilité** : Concerne les limites de ce qui peut être résolu par un ordinateur. Elle répond aux questions : Existe-t-il des tâches pour lesquelles il n'existe aucun algorithme ? Et étant donnée une tâche, peut-on dire s'il existe un algorithme qui la résolve ?
3. **Correction** : Concerne la validité et la conformité du code. Elle répond à la question : Peut-on être sûr qu'un algorithme réponde exactement au problème pour lequel il a été conçu ?

Question 3 (0.75 points)

Parmi les propositions suivantes concernant les déclarations de variables et de constantes en pseudo-code, lesquelles respectent la syntaxe et les concepts présentés dans le cours ?

- Variable val : entier (correcte)
- Constante MAX : entier ? 10 (correcte)
- Constante DEUXFOISMAX : entier ? MAX x 2 (correcte)
- Variables nom, prénom : chaînes de caractères (correcte)
- Constante TVA : réel = 20.6

Question 4 (2.00 points)

Examen

Algorithmique 1

Dans le cours, l'algorithme "CaDoitEchanger?" tente d'échanger les valeurs de deux variables réelles 'valA' et 'valB' de la manière suivante :

valA ? valB

valB ? valA

Expliquez pourquoi cette méthode échoue et décrivez précisément comment résoudre ce problème en détaillant l'utilisation d'une variable supplémentaire.

Réponse correcte : Cette méthode échoue car dès la première affectation (valA ? valB), la valeur d'origine contenue dans 'valA' est définitivement écrasée et perdue. Lorsque la deuxième instruction est exécutée (valB ? valA), la variable 'valB' reçoit en réalité sa propre valeur d'origine qui venait d'être copiée dans 'valA'. À la fin de cette séquence, les deux variables possèdent la même valeur (celle de 'valB' au départ).

Pour corriger ce problème, il est nécessaire d'introduire une variable temporaire (par exemple 'valTemp') pour sauvegarder l'une des valeurs avant l'écrasement. La séquence correcte est :

- 1. valTemp ? valA (on sauvegarde la valeur originale de valA dans le stockage temporaire)*
- 2. valA ? valB (on peut maintenant écraser valA avec la valeur de valB)*
- 3. valB ? valTemp (on affecte à valB la valeur originale de valA préalablement sauvegardée)*

Question 5 (1.25 points)

Considérons la structure alternative à choix multiples « SELON » et ses équivalents conditionnels présentés dans le cours. Quelles affirmations sont correctes ? (Plusieurs réponses possibles)

- L'utilisation de structures alternatives imbriquées permet d'arrêter l'évaluation dès qu'une condition vraie est rencontrée (structures exclusives). (correcte)
- L'utilisation de structures conditionnelles séquentielles successives (plusieurs SI ... FSI indépendants) oblige la machine à évaluer toutes les conditions, même si l'une d'elles a déjà été validée. (correcte)
- L'ordre des conditions imbriquées peut influencer le nombre de tests exécutés pour traiter un ensemble de données. (correcte)
- L'utilisation de SI séquentielles sans SINON produit exactement le même comportement que la structure imbriquée dans tous les cas, sans instructions supplémentaires.

Question 6 (4.00 points)

En vous basant sur la diapositive 33 ("TO DO"), calculez le nombre total d'instructions de comparaison (les tests d'égalité) qui seront évaluées lors de l'exécution pour un ensemble de 26 personnes composé de 24 étudiants (abréviation "M") et de 2 étudiantes célibataires (abréviation "Mlle"), dans les deux configurations suivantes :

- 1) L'exemple 2 (Diapositive 30 : structure imbriquée classique "M", "Mlle", "Mme").
- 2) L'exemple 4 (Diapositive 32 : structures "SI ... ALORS ... FSI" séquentielles).

Détaillez clairement vos calculs.

Réponse correcte : Voici le calcul détaillé du nombre de tests (comparaisons d'égalité) exécutés par la machine :

*1) **Configuration 1 : Exemple 2 (Structure imbriquée - Slide 30)***

*- **Pour un étudiant ("M")** :*

- Le premier test évalué est `abréviation = "M"`. Comme le résultat est VRAI, le programme exécute l'action associée et ignore tous les blocs `sinon` suivants.

- Nombre de tests par étudiant "M" = 1 test.

- Total pour 24 étudiants = $24 \times 1 = 24$ tests.

*- **Pour une étudiante célibataire ("Mlle")** :*

Examen

Algorithmique 1

- Test 1 : `abrégiation` = "M" -> FAUX. Le programme passe au bloc `sinon`.
- Test 2 : `abrégiation` = "Mlle" -> VRAI. Le programme exécute l'action et s'arrête là.
- Nombre de tests par étudiante "Mlle" = 2 tests.
- Total pour 2 étudiantes = $2 \times 2 = 4$ tests.
- **Total cumulé Configuration 1** = $24 + 4 = 28$ tests.

2) **Configuration 2 : Exemple 4 (Structures séquentielles - Slide 32)**

Dans cette structure, chaque bloc "SI ... FSI" est indépendant. La machine doit impérativement évaluer toutes les conditions de chaque bloc, quel que soit le résultat des blocs précédents.

- Chaque individu (qu'il soit "M" ou "Mlle") subira systématiquement les 3 tests de comparaison de la séquence (comparaison avec "Mme", puis avec "Mlle", puis avec "M").
- Nombre de tests par individu = 3 tests.
- **Total cumulé Configuration 2** = $26 \text{ individus} \times 3 = 78$ tests.

Question 7 (1.00 points)

Concernant la boucle « POUR » et la boucle « TANT QUE », quelles affirmations sont sémantiquement correctes d'après le cours ?

- Il est interdit de modifier la variable de contrôle de la boucle (le compteur) à l'intérieur du corps d'une boucle « POUR ». (correcte)
- La boucle « POUR » est privilégiée lorsque le nombre d'itérations est connu à l'avance. (correcte)
- Toute boucle « POUR » peut être réécrite de manière équivalente sous la forme d'une boucle « TANT QUE ». (correcte)
- Une boucle « POUR » évalue sa condition d'arrêt uniquement après avoir exécuté au moins une fois ses instructions.

Question 8 (3.00 points)

Écrivez en pseudo-code un algorithme demandant à l'utilisateur de saisir un entier qui doit obligatoirement être strictement supérieur à 0 et pair. Utilisez la structure « RÉPÉTER ... TANT QUE » pour contrôler cette saisie. Expliquez ensuite pourquoi cette structure est particulièrement adaptée à cet usage par rapport à une boucle « TANT QUE » classique.

Réponse correcte : Voici la proposition d'algorithme en pseudo-code :

...

Algorithme SaisieControlee

Variables

 valeur : entier

Début

 Répéter

 Afficher("Veuillez saisir un entier strictement positif et pair : ")

 Saisir(valeur)

 Tant que (valeur <= 0) ou (valeur % 2 != 0)

 Afficher("Saisie correcte ! Vous avez entré : ", valeur)

Fin

...

Explication de l'adéquation de la structure :

La boucle « RÉPÉTER ... TANT QUE » exécute toujours le corps de son traitement (la demande de saisie et la lecture) au moins une fois avant d'évaluer la condition d'arrêt. C'est idéal pour un contrôle de saisie car la variable à tester n'existe ou n'est renseignée qu'après cette première exécution.

Si on utilisait une boucle « TANT QUE », on serait obligé de dupliquer l'instruction de saisie en dehors de la boucle (phase d'amorçage) pour initialiser la variable avant le premier test, puis de la répéter à l'intérieur de la boucle (phase

Examen

Algorithmique 1

de relance). La boucle « RÉPÉTER » évite cette duplication de code.

Question 9 (1.00 points)

Dans quelle(s) situation(s) est-il préférable de choisir une boucle « TANT QUE » plutôt qu'une boucle « POUR » ? (Sélectionnez toutes les réponses valides d'après les principes du cours)

- Lorsque la boucle doit s'arrêter suite à un événement imprévisible, comme une saisie utilisateur spécifique (valeur sentinelle ou d'arrêt). (correcte)
- Pour effectuer un parcours exhaustif de tous les éléments d'un tableau de taille fixe connue.
- Lorsque la condition de continuation dépend d'un calcul complexe impliquant plusieurs variables mises à jour dynamiquement. (correcte)
- Pour exécuter une suite d'instructions un nombre de fois prédéterminé par une constante.

Question 10 (3.00 points)

Dans les diapositives 49 et 50, le cours aborde le problème suivant : « Saisir des données et s'arrêter dès que leur somme dépasse 500 ».

Comparez la structure de l'implémentation utilisant la boucle « RÉPÉTER ... TANT QUE » et celle utilisant la boucle « TANT QUE ... FAIRE ». Analysez la différence majeure entre ces deux approches concernant la phase d'amorçage.

Réponse correcte : D'après les diapositives du cours, les deux structures se présentent ainsi :

****Version avec RÉPÉTER (Slide 49/50) :****

```
...  
somme ? 0  
Répéter  
  Saisir(val)  
  somme ? somme + val  
Tant que somme <= 500  
...
```

****Version avec TANT QUE :****

```
...  
somme ? 0  
Saisir(val) {Amorçage indispensable}  
somme ? val  
Tant que somme <= 500 faire  
  Saisir(val) {Relance}  
  somme ? somme + val  
ftq  
...
```

****Analyse de la différence majeure (l'amorçage) :****

La différence réside dans le moment où le test de la condition est effectué :

- Avec la boucle ****Répéter****, le test de condition (`somme <= 500`) est évalué après le traitement. L'initialisation se limite à mettre la somme à 0, et l'entrée dans la boucle se fait naturellement.
- Avec la boucle ****Tant que****, le test est effectué dès l'entrée de la boucle. Pour que la condition ait un sens et puisse être évaluée au premier passage, il faut obligatoirement réaliser une première lecture et une première affectation à l'extérieur de la boucle (c'est l'****amorçage****). Ensuite, pour poursuivre l'itération, on doit rajouter une nouvelle saisie à l'intérieur de la boucle (la ****relance****). L'implémentation avec « Tant que » requiert ainsi une duplication de l'instruction `Saisir(val)` qui n'est pas nécessaire avec le « Répéter ».